

COMO CONSTRUIR UM AGENTE DE IA

Demonstração ao Cliente

As 8 etapas do método aplicadas ao assistente técnico do Saturno-Calc / MYDAS

Dimastec

Para o CTO · Natanael Moraes

Sousa Lima Consultoria · Everton Lima · 29 de agosto de 2026

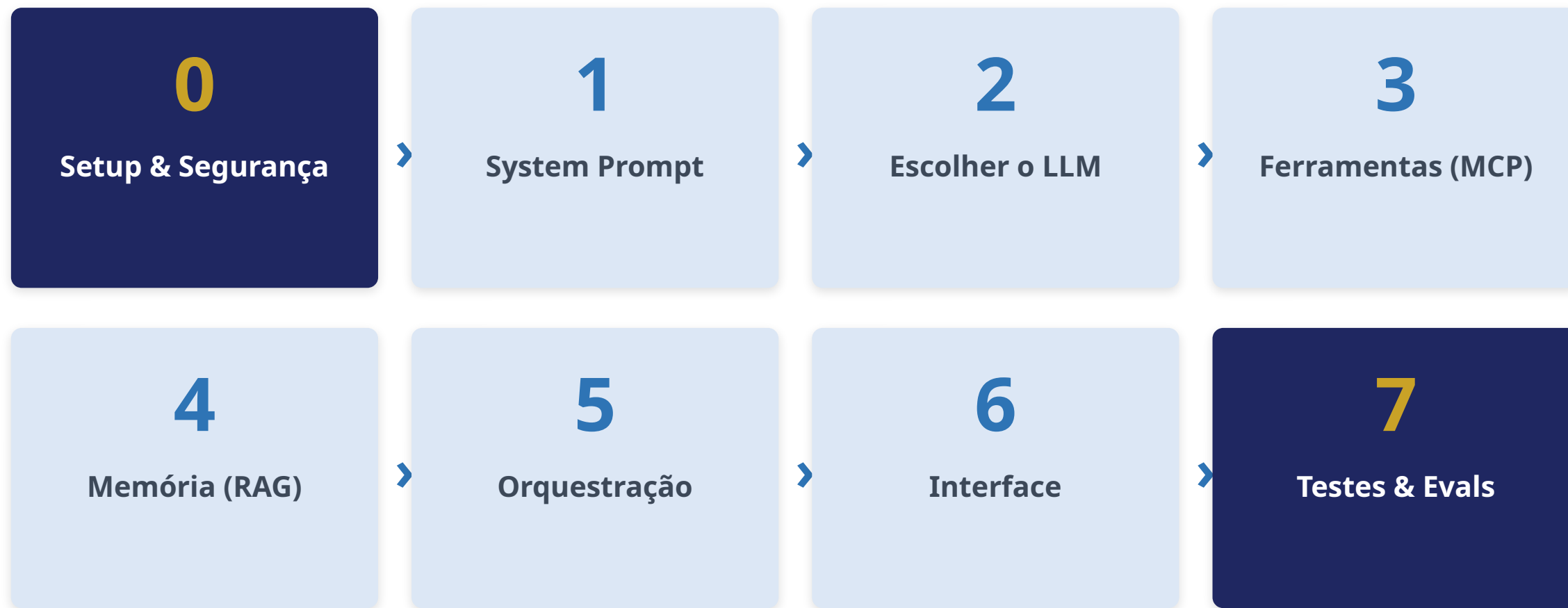
Objetivo e agenda da sessão

RESULTADO ESPERADO

- 1 CTO vê um agente em funcionamento aplicado ao contexto real da Dimastec — não um guia teórico.
- 2 Valida o método das 8 etapas como padrão para os demais casos de uso.
- 3 Fecha um próximo passo concreto: o piloto do assistente técnico read-only.

- | | | | |
|---|--------|-----|------------------------------------|
| 1 | 10 min | min | Contexto: o método das 8 etapas |
| 2 | 25 min | min | Demo ao vivo do assistente técnico |
| 3 | 10 min | min | Governança e guardrails na prática |
| 4 | 10 min | min | Evals e como medir qualidade |
| 5 | 5 min | min | Próximos passos e piloto |

As 8 etapas do guia em um só fluxo



Ordem pensada para valor rápido (MCP + RAG respondem no dia 1) e segurança antes de escala (guardrails e evals desde o início).

Setup, escopo e barreiras de segurança

0

COMO FAZER COM CLAUDE CODE

Defina o propósito em 1 linha: tirar dúvidas do time sobre o Saturno/MYDAS lendo ADRs e código — sem escrever e sem tocar produção.

Defina o critério de sucesso: resposta certa + cita a fonte de onde tirou.

Fixe os guardrails mínimos: proibido escrever, proibido dados de produção, sempre citar fonte. Classifique o caso como IA aplicada (não-dados sensíveis).

Revise com o CTO os limites antes de qualquer código.

REFERÊNCIA · ASSISTENTE-OS

- Cada soul declara propósitos e permissões em config.
- O runtime aplica allowlist de ferramentas e nega o resto por padrão (fail-closed).
- Alinhado à POL-IA-001 e à classificação de casos de uso da Dimastec.

System Prompt — o comando que define o agente

COMO FAZER COM CLAUDE CODE

No Claude Code, o prompt mora no arquivo de instruções do projeto/agente (ex.: AGENTS.md ou o prompt do subagente).

Persona: arquiteto sênior de Java que conhece o Saturno/MYDAS e responde em português com fonte.

Instruções: explicar a migração procedure → Java, citar camadas/patterns do hub (DAO, @Secured, route handler), responder só com base no material indexado.

Guardrails visíveis: nunca escrever, nunca acessar produção, dizer “não sei” se o RAG não achar, citar a fonte.

REFERÊNCIA · ASSISTENTE-OS

- Pool de prompts versionado (prompt-garden) — montagem determinística.
- Persona no início do prompt; sessão/RAG/histórico na cauda.
- Evita alucinação ao restringir a base de conhecimento.

Escolher o modelo (LLM) com visão de custo

2

COMO FAZER COM CLAUDE CODE

Documentação densa (ADR + código) pede janela de contexto ampla e bom raciocínio.

Compare 2-3 modelos antes de fixar: custo por 1M de tokens (entrada/saída), janela, latência e qualidade em PT-BR técnico.

No Claude Code, decida o modelo do agente e acompanhe o custo do uso.

Entregável: tabela com 2 modelos + decisão registrada em 1 ADR.

REFERÊNCIA · ASSISTENTE-OS

- O roteador escolhe o modelo por tier e mede o custo por chamada.
- Custo imutável rastreado por chamada (kernel.db).
- Transparência de custo para o cliente desde a escolha.

custo/1M tokens

janela de contexto

ADR

Ferramentas & Integrações (MCP)

3

COMO FAZER COM CLAUDE CODE

A Dimastec já tem servidor MCP interno — conecte-o agora.

Exponha ferramentas somente-leitura: busca na base de conhecimento (RAG), leitura de ADR, leitura de código.

No Claude Code, as ferramentas MCP entram no mesmo arquivo de configuração do projeto.

Teste ao vivo: uma chamada MCP que busca um trecho de ADR e responde citando o arquivo.

REFERÊNCIA · ASSISTENTE-OS

- Kernel expõe ferramentas MCP sobre stdio com classificação L1/L2/L3.
- A soul só enxerga as ferramentas da sua allowlist.
- Zero Trust: ferramentas de escrita bloqueadas por padrão.

MCP

read-only

allowlist

Memória (RAG + histórico)

4

COMO FAZER COM CLAUDE CODE

O assistente precisa lembrar do contexto sem ter tudo no prompt a cada vez.

Indexe documentos aprovados (allowlist): ADRs do Saturno, políticas (ISO 27001/LGPD/POL-IA-001) e código sanitizado — sem dados de produção.

Persista o histórico por thread/sessão para conversas contínuas.

Quando o RAG não acha nada, responder “sem contexto” em vez de inventar.

REFERÊNCIA · ASSISTENTE-OS

- RAG com embeddings + re-rank opcional e cache semântico.
- Limiar de relevância: sem doc relevante, responde sem contexto.
- Política de memória impede indexar dados sensíveis.

vetorial

allowlist

sem dados sensíveis

Orquestração — o fluxo do agente

5

COMO FAZER COM CLAUDE CODE

Defina o caminho: entende a pergunta → recupera contexto (RAG) → decide se usa ferramenta → responde.

Modos por complexidade: pergunta curta → resposta rápida; “passo a passo”/“análise” → fluxo mais profundo.

Decida quando ele pode chamar ferramenta e o que fazer em erro (limite de tentativas, timeout, dizer “não consegui”).

REFERÊNCIA · ASSISTENTE-OS

- Grafo LangGraph: retrieve → generate ↷ tools com limite de iterações.
- Streaming por etapa (cada node emite evento).
- Escalonamento quando o contexto é fraco.

retrieve

generate

tools

retry

Interface — como o time vai usar

6

COMO FAZER COM CLAUDE CODE

Fase 1: chat no terminal/IDE (Claude Code) sendo o próprio assistente, com prompt + MCP configurados.

Fase 2: interface web de chat para o time, com histórico por usuário.

Fase 3 (opcional): API para integrar ao fluxo de suporte existente.

Comece simples; evolua a interface conforme uso real.

REFERÊNCIA · ASSISTENTE-OS

- Chat via REST/WebSocket com streaming etapa-a-etapa.
- Painel web mostra o grafo rodando ao vivo.
- Ideia para o CTO: ver o agente “pensar” em tempo real.

terminal/IDE

web

API

Testes & Evals — qualidade mensurável

COMO FAZER COM CLAUDE CODE

Sem eval não há como provar que o assistente melhora ao longo do tempo.

Monte um golden set real do Saturno: ex. “qual o padrão de DAO multi-tenant?” → citar o ADR/pattern; “todo endpoint precisa de @Secured?” → sim, pelos rules.

Meça hit@1 (doc certo em 1º?), MRR e recall@5 + fidelidade ao contexto (não alucinar).

Gate no CI: pergunta abaixo do piso (ex.: $\text{hit@1} \geq 0.7$) bloqueia o release.

REFERÊNCIA · ASSISTENTE-OS

- Comando de avaliação de RAG com golden set (hit@1/MRR/recall).
- Sai com erro se o piso não for atingido — gate em CI.
- A Dimastec já tem golden set com 23 casos no hub — ponto de partida.

golden set

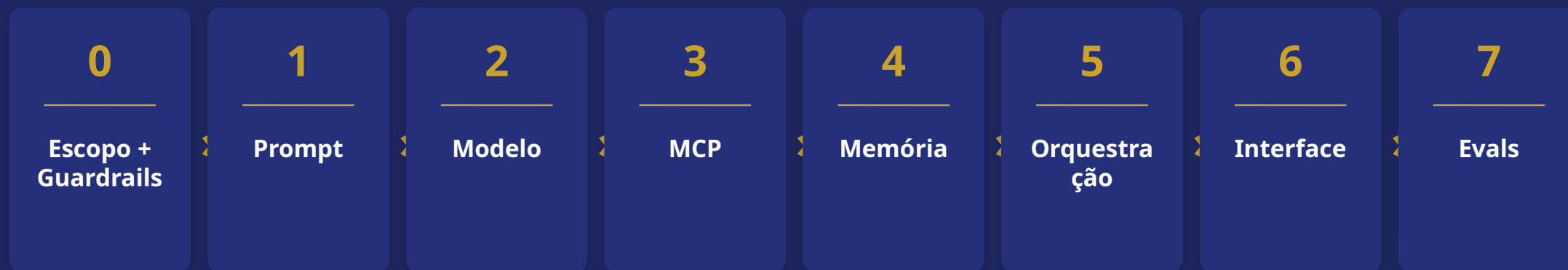
hit@1

MRR

gate CI

8 ETAPAS EM 7 MOVIMENTOS

A ordem certa para o CTO ver valor rápido e segurança desde o início



Cada etapa fecha com um artefato mínimo: ADR · prompt versionado · tabela de modelo · allowlist · golden set.

Mensagens-chave para o CTO

1 Não é teoria

É o mesmo fluxo que aplicamos no Saturno-Calc: propósito → LLM → ferramentas → memória → orquestração → evals.

2 Já em andamento

MCP e Projeto Zero estão rolando; a Dimastec já usa Claude Code, Rovo e servidor MCP interno — a demo formaliza isso.

3 Governança embutida

Cada etapa respeita ISO 27001 / LGPD / POL-IA-001 e a classificação de casos de uso (sem dados biométricos em runtime).

4 Porta de entrada

Esta demo abre caminho para o piloto do assistente técnico do Mês 3 da mentoria.

PRÓXIMOS PASSOS

Do piloto à escala, com prazo e dono

- | | | | | |
|---|---------------------------------|---|------------------|--------------------|
| 1 | Aprovar o piloto | Assistente técnico read-only no escopo do Mês 3 da mentoria | 1 semana | Everton + Natanael |
| 2 | Definir fontes aprovadas | Allowlist de documentação, ADRs e tickets sanitizados | 2 semanas | Everton |
| 3 | Métricas | Baseline de qualidade e fidelidade do assistente | 2 semanas | Everton + Natanael |
| 4 | Treinar multiplicadores | Times de engenharia no método das 8 etapas | Contínuo | Sousa Lima |

Obrigado — vamos construir juntos.